The Promotional Plan Optimiser

A Plain-English Guide for Merchants — What It Is, and How It Works, One Step at a Time

Albertsons / Line 8 • A joint merchandising + data-science build • Working Reference

Summary

This guide explains how the promotional optimiser works and answers the questions merchants ask most. It is a **joint build** between the merchandising and data-science teams — it encodes merchant judgement, it does not replace it. It follows the **six steps** of an actual run, with the questions grouped under the step they belong to. The themes that matter most: the optimiser only ever proposes promotions that have actually been run; it predicts sales and profit, then chooses the best combination for the goal you pick while protecting the other goals and respecting business rules; those rules are **discovered from the data** and reflect established merchandising practice; and the problem is genuinely division-scale — tens of millions of options — which is why a serious optimiser is needed.

1. Start Here: the Words You Will Hear

Term	What it means (plain English)
The Optimiser	The tool that decides the best promotion to run on each item, in each week, for the goal you choose. It picks from a menu of real options and respects business rules.
The CRF / demand model	A trained prediction engine (a neural network) that answers 'if we run *this* promotion on *this* item, how many units and how many sales dollars will we get?' It learned from years of real sales.
A candidate	One possible promotion for one item in one week (e.g. '20% off jalapeños, week 3'). Every candidate is copied from something that actually happened.
A promo tactic	The *type* of one promotion — a percentage off, a price point ($3.99), a buy-one-get-one, a digital coupon — on one item.
A plan	The *whole set* of tactic choices across every item and week. One plan = thousands of individual decisions that hang together and respect the rules.
The solver	The mathematical engine that sifts an astronomically large number of possible plans and finds the best one that obeys every limit.
A guardrail	A business rule that steers the optimiser away from an unhealthy choice by attaching a penalty — without banning it outright.
Pruning	A hard filter that removes clearly out-of-bounds options *before* the solver runs.

2. The Whole Run in Six Steps

The rest of this guide follows these six steps in order. Each step contributes to landing on a plan that is both high-performing and defensible.

Step	What happens	Covered in
1. Build the candidate menu	Gather the realistic promotions worth testing — all from history.	Section 3
2. Score each candidate	Predict units & revenue (CRF); compute profit; adjust for reliability.	Section 4
3. Learn the limits & relationships	Discover each category's caps and floors, and which items interact.	Section 5
4. Prune	Hard-remove out-of-policy options; never leave an item with nothing.	Section 6
5. Solve & score	Combine goals, reliability and guardrails into one score; pick the best plan.	Section 7
6. Choose & refine	The five best plans per goal (1 optimum + 4 alternatives); merchant selects, edits, repairs.	Section 8

What This Guide Answers

Every question in this guide, grouped by step. Jump to whichever applies to you.

Step 1 — Building the Candidate Menu (Section 3)

- **Q3.1** Where do the promotions you test actually come from?
- **Q3.2** Could it ever suggest a discount we have never used on that item?
- **Q3.3** If a tactic is learnt in one place, can it be applied somewhere else? (How grounded is it, really?)
- **Q3.4** Why not let a clever engine design the best promotion from scratch?
- **Q3.5** Can we still experiment with genuinely new tactics?

Step 2 — Scoring Each Candidate: the Demand Model (Section 4)

- **Q4.1** What does the demand model (CRF) actually predict?
- **Q4.2** Does the model predict profit (AGP) too?
- **Q4.3** What is ‘reliability’ and how does it change the numbers?

Step 3 — Learning the Limits and Relationships (Section 5)

- **Q5.1** Did you just invent the rules? Where do they come from?
- **Q5.2** What exactly is learnt, and is any of it per category?
- **Q5.3** What are interaction effects, and how does the optimiser use them?
- **Q5.4** Can I cap how many promotions a vendor runs, and keep their deals from killing each other?

Step 4 — Pruning: the Hard Filters (Section 6)

- **Q6.1** Which options get removed before the solver even runs?

Step 5 — Solving: Turning Everything Into One Score (Section 7)

- **Q7.1** How do units, sales, profit, reliability and guardrails come together into a single score?
- **Q7.2** So to choose a promotion, what do I actually look at — Units, Sales and Profit, or the guardrails and reliability?
- **Q7.3** You keep saying a guardrail ‘charges’ an option. Charges what, and by how much?
- **Q7.4** Some guardrails are written as formulas like ‘depth² × 8’. What does that mean?
- **Q7.5** Can I simply switch guardrails off, category by category?
- **Q7.6** Shouldn’t we just run the optimisation with no rules first, to see the ‘pure’ answer?
- **Q7.7** Why are the scenarios pre-generated for me, instead of run live when I flip a switch?
- **Q7.8** How big is this problem, really?
- **Q7.9** So why is it slow, in plain terms — and what is this ‘Gurobi’?
- **Q7.10** Why that engine, and not a free one?

Step 6 — Choosing and Refining Your Plan (Section 8)

- **Q8.1** What do I actually receive, and how do I choose — is it just ‘pick a goal’?
- **Q8.2** What is the ‘repair-and-score’ step, and what happens if I create my own custom promotion?

STEP 1

3. Building the Candidate Menu

Q3.1. Where do the promotions you test actually come from?

A. For every item and week, the tool assembles a short menu of **candidates** from **five grounded sources**, all drawn from real history: **(1)** no promotion; **(2)** the current plan; **(3)** last year, same week; **(4)** this item's own promotions over the last ~53 weeks; and **(5)** its category's promotions over the last ~53 weeks. It keeps up to ~20 options per item-week, ranked by relevance. Every depth and tactic is a copy of something that genuinely happened.

Q3.2. Could it ever suggest a discount we have never used on that item?

A. No. Because the menu is built only from observed events, it cannot dial in a 'creative' depth just because the math likes it. Every recommendation traces back to a promotion that has actually run.

ILLUSTRATION — Jalapeños

If jalapeños have only ever been promoted at 15% and 20% off, the menu is: no promotion, 15% off, 20% off, last year's actual deal, plus a few options seen elsewhere in Produce. It will never propose a 40%-off jalapeño, because nothing like that exists in the history.

Q3.3. If a tactic is learnt in one place, can it be applied somewhere else? (How grounded is it, really?)

A. This is worth being precise about, because the two history sources behave differently:

- **The item's own history is store-level.** Source (4) is tied to that item in that store — the tightest possible grounding, no spread.
- **The category history is pooled within the division.** Source (5) deliberately gathers tactics seen across the category, division-wide, so an item with thin history of its own still gets sensible options from its category peers. So a Produce tactic *can* appear as a candidate for another Produce item in the same division — by design, to avoid starving new or quiet items.
- **Nothing crosses divisions.** All history is filtered to the item's own division, and item **interactions** (Section 5) are scoped to the price area. A tactic from one division is never offered in another.

Q3.4. Why not let a clever engine design the best promotion from scratch?

A. An earlier design did exactly that — a **free-search** optimiser that treated depth as a continuous dial and invented combinations. We moved away from it for three reasons: it **invents unfounded tactics** (a depth or deal type an item has never carried, hard to defend); it **can be gamed** (it optimises against the model, not the real world, and finds options that look great on paper but fail in store); and it is **hard to audit**. The grounded-menu approach keeps the intelligence but stays explainable.

Q3.5. Can we still experiment with genuinely new tactics?

A. Yes, but honestly it is a staged process, not a quick toggle — because the **demand model has to learn the new tactic first**. The CRF recognises promo *type* as a known input; a type it has never seen has no learned behaviour and cannot be scored reliably until the model is **retrained** to include it. The path is: **pilot → gather real response data → retrain the CRF and re-learn the limits → add the tactic to the standard menu**. After that it is treated like any other grounded option.

STEP 2

4. Scoring Each Candidate: the Demand Model

Q4.1. What does the demand model (CRF) actually predict?

A. For every candidate it predicts **expected units** and **expected revenue (sales dollars)** — how much that item would sell under that promotion. It is a neural network trained on years of real sales and promotions, and it runs in parallel across many machines, so even millions of candidates are scored quickly.

Q4.2. Does the model predict profit (AGP) too?

A. No — and this is an important detail. **AGP (profit) is calculated outside the model**, from the predicted revenue and units plus the item's true cost:

> *AGP = predicted revenue – (predicted units × dead-net cost per unit)*

'Dead-net cost' is the real landed cost of a unit after vendor allowances and deductions, so AGP is the profit left after the cost of the goods actually sold. The model supplies the sales side (units and revenue); the cost side comes from cost data, and profit is the difference.

Q4.3. What is 'reliability' and how does it change the numbers?

A. Some items respond to a deal the same way every time; others are hit-or-miss. **Reliability** is simply how consistent an item has been in the past. When an item's response is steady, the tool takes its forecast at face value; when it is hit-or-miss, the tool is more **cautious** and won't lean too hard on a rosy number it can't count on. Crucially, this caution only affects **which tactic the tool leans toward recommending** — the Units, Sales and Profit forecast you actually see on screen is unchanged.

STEP 3

5. Learning the Limits and Relationships

Q5.1. Did you just invent the rules? Where do they come from?

A. No — they come from two solid places:

- **Measured from the actual data.** Most limits are read straight off the real sales and promotion record — the margin floor is the level the category's own promotions have historically held; the depth ceiling is the deepest the category has actually gone; the promo-intensity cap is how heavily the category has really been promoted. Setting sensible operating ranges from real history is a standard, well-understood method, and it means the rules describe *what already happens*, not an opinion.
- **Recognised category-management concepts, shaped with the merchandising team.** Ideas like protecting price image on key items, keeping vendor funding on plan, pacing deep deals, and not over-loading the circular are well-established concepts in promotional planning. We did not decide on our own which ones matter or how strict they should be — they were chosen and calibrated together with merchandising, and they stay configurable so the business owns them.

Where a rule needs a starting number before there is enough clean data to learn it, we use a clearly-labelled **coded policy default** (see the policy-numbers table in Section 7) — a transparent placeholder, designed to be replaced by a learnt value as data accumulates.

Q5.2. What exactly is learnt, and is any of it per category?

A. Yes — the most important limits are learnt **per category**, so a price-sensitive category and a deal-driven one are treated differently, automatically. Margin floors, depth ceilings and promo-intensity caps are learnt per category; vendor limits are learnt per vendor; ad capacity per price area and week; and item interactions per price area. The full list of guardrails (with which are category-specific) is in Section 7.

Item interaction effects (cannibalisation & halo)

Q5.3. What are interaction effects, and how does the optimiser use them?

A. Items don't sell in isolation. The tool learns relationships from real basket and switching data and feeds the most important into the decision:

- **Substitutes (cannibalisation):** promoting one item pulls sales from a close cousin — a **negative** effect. The solver will not double-count 'stolen' volume and may avoid promoting both deeply the same week (own-brand substitutes pull harder).
- **Complements (halo):** items bought together (chips and salsa) — a **positive** effect, **currently shown in reporting** to inform you, but not yet inside the solver's math.

To stay focused and fast, each candidate keeps only its **two strongest** relationships in the solve (the rest are dropped), each item is limited to its top few relationships overall, and their total pull is capped so they steer the plan without dominating it. Relationships are **scoped to the price area**.

ILLUSTRATION — Cannibalisation in action

If Coke 12-pack and Pepsi 12-pack are learnt substitutes and you deeply promote Coke, the solver knows some of Coke's 'extra' units are really pulled from Pepsi. It won't double-count that volume, and it may avoid running both deep the same week — a more honest plan than treating each in isolation.

Q5.4. Can I cap how many promotions a vendor runs, and keep their deals from killing each other?

A. The 'don't kill each other' part is largely handled today: the interaction effects above de-conflict competing deals (the solver spaces two cannibalising items out, or softens one), and pacing guardrails (Deep-Week Gap, Cross-Item Pacing, Vendor Funding Ramp) add spacing and limit week-over-week vendor ramp.

ROADMAP / LATEST BUILD — Vendor event count

An explicit control to cap the **number of events** a vendor may run, and to actively trade depth between competing events, is part of the latest build and not yet covered here. Today, spacing comes from the interaction effects and pacing guardrails rather than a hard per-vendor event count.

STEP 4

6. Pruning: the Hard Filters

Q6.1. Which options get removed before the solver even runs?

A. Five hard filters remove clearly out-of-bounds candidates. Most thresholds are learnt per category; if an item is left with no option, a **rescue** step puts the safest one back so no item is ever dropped entirely.

Pruning rule	Removes (with an example)	Active?
Margin Floor **[category-specific]**	Promos whose profit rate falls below the category's historical floor — e.g. a milk deal so deep it would sell below the dairy profit floor.	Yes
Vendor Min Commit	Options where vendor funding is far below the vendor's minimum — e.g. a deal the vendor has barely funded.	Yes
Role Depth Cap	Options deeper than this item-role normally runs — e.g. a background item pushed to a headline depth.	Yes
Max Absolute Depth **[category-specific]**	Options deeper than the category has ever gone (plus a small buffer) — e.g. 70% off where the category has never passed 50%.	Yes
Depth Jump Cap	Options that lurch far deeper than the item's own recent baseline in one step — e.g. 10% last week to 45% this week.	Yes

STEP 5

7. Solving: Turning Everything Into One Score

Q7.1. How do units, sales, profit, reliability and guardrails come together into a single score?

A. This is the heart of it. For every candidate, the solver builds one number, then picks the combination of candidates (one per item-week) that makes the plan's total as high as possible. The number is built like this:

Ingredient	What it contributes to a candidate's score
1. The three goals	The predicted Units, Revenue and AGP gains, blended in **priority order** — the goal you chose dominates, the other two count far less but still count.

Ingredient	What it contributes to a candidate's score
2. Reliability	If the item's past response is hit-or-miss, its contribution is **trimmed back**, so the tool leans on steadier bets. *(The units/sales/AGP you see stay raw.)*
3. Minus guardrail charges	Every guardrail the option breaks **subtracts** points (see 'What a guardrail charge means' below). A clean option is charged nothing.
4. Minus interaction & over-spend effects	Cannibalisation/halo and any over-budget spend are **subtracted** here too — but **not** from each tactic's reported numbers.

Put simply, the tool ranks each option on a **total score**: start with how much it helps your chosen goal (the other two goals count a little too), **trim it back** where the prediction is shaky, then **subtract** the guardrail charges and any cannibalisation. That total is used only to **decide which tactic to recommend** — it is not a rewrite of the numbers. The Units, Sales and Profit you see stay the **raw forecast**, and the guardrail charges are shown **separately**. So an option with higher raw sales won't automatically win: if its total comes out lower once shaky predictions and rule-health are counted, a cleaner option wins instead.

Q7.2. So to choose a promotion, what do I actually look at — Units, Sales and Profit, or the guardrails and reliability?

A. First, an important distinction. The Units, Sales and Profit numbers you see are the **raw forecast from the demand model** — they are **not** rewritten by guardrails, reliability or interactions. A tactic that shows higher sales really is forecast to sell more, on its own.

But the tool does **not** automatically pick the highest-sales tactic. Which tactic it **recommends** is decided by the **total score** above, which layers reliability, guardrail charges and interaction effects on top of that raw forecast. Those layers shape the **ranking**, not the displayed numbers — so a high-sales option that strains the rules or leans on a shaky prediction can be out-ranked by a cleaner one. You therefore read the honest raw trade-offs (Units vs. Sales vs. Profit, vendor spend, deep weeks), with the guardrail charges shown alongside as a separate line, and trust that the option put forward already accounts for rule-health and reliability through its score (Section 8 covers comparing the alternatives).

Why not just subtract the raw guardrail penalties? Because those raw numbers are on very different scales and units — dollars of unfunded markdown, plain points, squared depths — so deducting them as-is would let one rule swamp the business goal, or a big-dollar item drown out a small one. Each charge is multiplied by a small, deliberate weight (tuned per goal) so guardrails act as **tie-breakers and gentle steering**, never overriding the goal you chose.

And why aren't interactions baked into the raw numbers? Because a tactic's knock-on effect on its neighbours depends on the **whole set** of promotions finally chosen — which isn't known until a plan is selected, and which you may change. So interactions steer **which tactic is recommended** but are deliberately kept out of each tactic's standalone numbers; the combined effect of the final set is worked out at the **repair-and-score** step (Section 8).

Every guardrail, what it is charged in, and the danger of not having it

Around twenty guardrails. Each adds a charge the solver must overcome. The **'Charged in'** column matters: some charges are real **dollars** (lost margin or unfunded spend), one is in **units**, and the rest are unitless **points** (depth-based formulas). Because these are genuinely different units, each is multiplied by its small weight before they are combined — which is exactly why we never just add the raw numbers (Q7.2). The shaded column is the danger — what the optimiser would do without the rule.

Profit & margin

Guardrail	What it charges	Charged in	DANGER if we don't have it
Profit Protection Floor **[category-specific]**	Promos whose profit rate drops below the category's learnt floor	**$**	Chasing units, the optimiser slashes a staple so deep it sells volume at a loss.
Allowance Floor	Promos where vendor allowance covers too little of the deal	**$**	Deep promos with thin trade support — the store carries the margin.

Guardrail	What it charges	Charged in	DANGER if we don't have it
Bleeder Protection	Projected profit drop on items that drain profit when promoted	$	'Bleeder' items get promoted and leak profit unnoticed.

Vendor funding

Guardrail	What it charges	Charged in	DANGER if we don't have it
Funding Guardrail	The unfunded markdown — deal cost the vendor does *not* cover	$	The store pays for discounts the vendor was meant to fund (e.g. $0.70/unit).
Vendor Min Commitment	Spend below the vendor's committed minimum	$	We under-deliver agreed vendor programs.
Vendor Funding Band	Spend that strays outside the vendor's normal band	$	Trade spend swings far above or below plan.
Vendor Funding Ramp	Big week-over-week jumps in vendor spend	$	Lumpy, hard-to-manage trade spend.

Price image & shopper trust

Guardrail	What it charges	Charged in	DANGER if we don't have it
Price Image Corridor	When a key-value item's price leaves its expected band	Points	KVI prices wander, eroding the 'low price' perception shoppers rely on.
Promo Pacing	A steady charge that grows with discount depth	Points	Items promoted constantly and deeply — shoppers learn to only buy on deal.
Deep-Week Gap	Deep weeks placed too close together	Points	Back-to-back deep weeks that pull demand forward and erode the baseline.
Depth Volatility	Big swings in depth week to week	Points	Prices whipsaw (10%, 40%, 5%, 35%…), confusing shoppers.
Deep Discount Risk	A sharp brake on discounts past ~35%	Points	Very deep discounts that rarely pay back.

Category & ad space

Guardrail	What it charges	Charged in	DANGER if we don't have it
Category Intensity **[category-specific]**	When too much of a category is on deal in one week	$	A whole category on promo at once — it cannibalises itself and trains shoppers.
Ad Capacity	When the circular / front page is over capacity	Points	More features than the ad can physically hold.
Cross-Item Pacing	Many deep items crowded into the same window	Points	Too many deep deals stacked in the same week.

Item interaction effects

Guardrail	What it charges	Charged in	DANGER if we don't have it
Substitute Pacing (off by default)	Too many deep weeks within a group of substitutes	Points	Close substitutes all run deep at once.
Complement Control (off by default)	Deep stacking of items bought together	$	Margin given away on complementary items at the same time.
Leader Protection	Projected revenue drop on a category leader	$	The item that anchors the category gets eroded.

Protecting the other goals

Guardrail	What it charges	Charged in	DANGER if we don't have it
Cross-Units Floor	Projected drop in **units** vs. no-promo	**Units**	A profit or sales plan that quietly sheds unit volume.
Cross-Revenue Floor	Projected drop in **revenue** vs. no-promo	**$**	A units or profit plan that quietly sheds sales dollars.
Cross-AGP Floor	Projected drop in **profit** vs. no-promo	**$**	A units or sales plan that bleeds margin.
Pain Budget	Excessive revenue loss on items allowed to be loss-leaders	**$**	Too much sacrificed on loss-leaders.

What a guardrail 'charge' means

Q7.3. You keep saying a guardrail 'charges' an option. Charges what, and by how much?

A. A charge is simply **points subtracted from that candidate's score**, so a charge of X means the option must earn at least X more on the goal to be worth keeping. Each charge has a **size** (how far it breaks the rule) and a **weight** (how much we care, which depends on the goal).

ILLUSTRATION — The Funding Guardrail charge, in dollars

*A 30%-off deal on a $4.00 item gives away $1.20 of margin per unit, but the vendor funds only $0.50. The **unfunded markdown** is $1.20 – $0.50 = **$0.70 per unit** — the charge's size. Under a Profit plan the funding weight is about 0.12, so the score is docked 0.70 × 0.12 ≈ **0.084 points per unit**. The deal can still win — but only if it earns more than that elsewhere. Fully-funded deals are charged nothing.*

Q7.4. Some guardrails are written as formulas like 'depth² × 8'. What does that mean?

A. It means the charge grows faster and faster as the discount deepens. Three depth-based guardrails behave like this (raw charge for one item, before the goal weight):

Discount depth	Promo Pacing (steady: depth × 5)	Depth Volatility (accelerating: depth² × 8)	Deep Discount Risk (brake past 35%)
10% off	0.50	0.08	0.00
20% off	1.00	0.32	0.00
35% off	1.75	0.98	0.00
50% off	2.50	2.00	2.70
60% off	3.00	2.88	7.50

- **Promo Pacing** rises in a straight line — a steady 'don't lean on promotions' nudge.
- **Depth Volatility** accelerates — almost free when shallow, biting harder the deeper you go.
- **Deep Discount Risk** is a brake — nothing until 35% off, then it climbs steeply.

Can I turn the rules off, or run with no rules?

Q7.5. Can I simply switch guardrails off, category by category?

A. Today, guardrails can be switched on or off individually and weighted per goal, but those switches act **globally**, not per category. What already is category-specific is the part that matters most: the **learnt caps differ by category**. Those learnt values can be **surfaced for review as an optional step in your 52-week planning process** — so you can see, and sign off on, each category's floor and ceiling.

Q7.6. Shouldn't we just run the optimisation with no rules first, to see the 'pure' answer?

A. An unconstrained run isn't the 'true' answer — it's an unusable one, for two reasons. First, some limits aren't guardrails at all but hard requirements (exactly one decision per item-week, vendor spend bands, a floor that stops total profit going backwards) — the problem is nonsensical without them. Second, with the guardrails off the optimiser chases the goal blindly. Two illustrations of what it actually returns:

ILLUSTRATION — Profit goal, margin floor switched off (illustrative)

*Maximising profit with no Profit Floor, the optimiser's answer for Dairy puts **milk at 40% off in 5 of the 6 weeks** — because the model sees the volume and nothing stops it, even though that destroys margin and price image. With the floor on, milk is paced and capped near what Dairy has historically run.*

ILLUSTRATION — Sales goal, no category or pacing rules (illustrative)

*Maximising sales with no Category Intensity or pacing rules, the optimiser puts **most of the Salty Snacks category on deep deal in the same circular week** — a big sales spike that cannibalises the category and trains shoppers to wait. With the rules on, the deals are spread across weeks and the category stays within its normal promo intensity.*

Q7.7. Why are the scenarios pre-generated for me, instead of run live when I flip a switch?

A. Because the solve is heavy (see 'The engine, and why a full run takes hours' below): a full run is hours of computation. We **pre-compute** the best plan and alternatives for each goal in batch, so you get instant results to browse and compare. If guardrails could be flipped **live**, each flip would require re-running that heavy solve before new results appeared — not instant. This is why on/off and weight changes are handled as a **planning-cycle step** (reviewed and applied in batch), not a real-time dial — and why the per-category learnt values are best reviewed during the 52-week plan.

The coded policy numbers — why they exist and how they become learnt

Policy default	Value	Why it exists	How it becomes learnt
Depth-jump limit	0.20	Don't let a deal jump >~20 points deeper than the item's recent baseline in one step.	Learn each category's typical week-to-week change.
Max-depth headroom	0.20	Allow ~20 points above the historical ceiling so legitimate stacked deals aren't blocked.	Learn how often real stacked deals exceed the ceiling.
Deep-promo threshold	0.30	Defines what counts as a 'deep' deal (30%+) for pacing/gap rules.	Learn the depth at which shopper stock-up behaviour changes, per category.
Vendor min-commit trigger	50%	Flag a deal whose vendor funding falls below half the committed minimum.	Learn each vendor's real shortfall tolerance.

The engine, and why a full run takes hours

Q7.8. How big is this problem, really?

A. This is not a one-category demonstrator — to be useful it must cover an **entire division (ROG): every category, every store**. The size comes from the core decision the tool repeats over and over: for **each item, in each week, choose one promotion from about 20 candidate options**.

The core decision the tool makes	Roughly
Items (NCRCs) per store	~31,000
× weeks in the plan	~6
= 'pick-one' decisions per store	**~186,000**
× candidate options per decision	~20
× stores in a division (ROG)	95 – 274

So a single store is roughly **186,000 separate pick-one decisions**, each among ~20 options — and a division has between 95 and 274 stores. The number of *possible plans* for just one store is 20 multiplied by itself 186,000 times — **more combinations than there are atoms in the universe**. They cannot be listed or checked by hand, which is why a real mathematical optimiser is required.

On top of that, the tool layers pairwise cross-item effects — how promoting one item changes its neighbours (cannibalisation and halo). Because items affect each other **in pairs**, the math becomes what optimisation specialists call

quadratic — markedly harder than choosing each item on its own. That overlay is grounded in a large body of real basket data; the table below is the scale we learn the cross-item effects from (it is interaction evidence, *separate* from the decision base above):

Division (ROG)	Stores	Baskets analysed	Item × promo combinations	Unique item × promo states
SHAW	25	17,517,745	14,202,796	760,936
AJWL (Jewel)	190	105,367,053	146,822,728	8,280,430
SNCA	274	168,825,736	169,993,019	9,102,692
AIMT	95	34,726,117	50,824,426	3,121,036
SDEN	127	54,335,409	69,609,920	4,261,534
Total	**711**	**380,772,060**	**451,452,889**	**25,526,628**

To keep that overlay workable, the optimiser keeps only each item's **two strongest** cross-item relationships rather than every possible pair. Even so, learning them means working through **hundreds of millions of baskets** — the evidence that the cross-item effects built into the plan are real, not guesswork. **This is exactly why a real mathematical optimiser is required.**

Q7.9. So why is it slow, in plain terms — and what is this 'Gurobi'?

A. Two different jobs happen. **Predicting sales** for every option is shared across many machines at once, so it is fast and **not** the hold-up. **Picking the best combination** is the hard part: it is done by a specialist optimisation engine (named Gurobi) that works through **one store at a time, in sequence** — like a single expert reviewing each store's entire plan, one after another, for hundreds of stores. That queue is what takes hours.

Q7.10. Why that engine, and not a free one?

A. The sticking point is the item interactions — 'these two items affect each other' is a harder kind of math (two decisions multiplied together). We looked at the alternatives:

Engine	What it is	Why we did not pick it
A free solver (CP-SAT)	An open-source optimisation engine	It can't take the 'two items affect each other' math directly — it must approximate it with lots of extra bookkeeping, which is slower and less exact at this scale.
A trial-and-improve search (NSGA-II)	A method that keeps tweaking a plan to improve it	It only *guesses* a good plan and can't prove it is the best, nor give a clean reason for each choice — weaker for an auditable merchandising plan.
Gurobi (chosen)	A commercial exact optimisation engine	It handles the interaction math exactly, returns the provably best plan, and is explainable — worth the licence at division scale.

The speed lever

The goals and the stores are **independent** of each other, so those jobs **could** run side by side on many machines to cut the wait sharply. Only the alternative plans within a goal must stay in order (each must differ from the last). The route to faster runs is **more machines working in parallel** — not fewer safeguards.

STEP 6

8. Choosing and Refining Your Plan

Q8.1. What do I actually receive, and how do I choose — is it just 'pick a goal'?

A. For each goal the optimiser delivers the **five best plans** — plan 1 is the optimum, and plans 2–5 are distinct, high-quality alternatives. It builds plan 2 by requiring at least one item to change from plan 1 and gently discouraging re-using the same choices, then re-solving — and so on — and it makes them **tactically diverse** (different mixes of deal types) rather than near-copies. Within each plan, every item also shows a short menu of its best alternative tactics.

Choosing is **not** just ‘pick a goal’. The goal narrows the field; then you compare the five plans on the trade-offs shown — units vs. sales vs. profit, vendor spend, how many deep weeks, the depth mix, and guardrail health — and pick the one that best fits your strategy and calendar. The tool’s job is to hand you a few genuinely strong, distinct options; the final call is a merchant judgement.

Q8.2. What is the ‘repair-and-score’ step, and what happens if I create my own custom promotion?

A. The **repair-and-score** step is how a chosen or edited plan gets tidied and re-estimated as a whole, rather than left as a set of standalone tactic forecasts. When you edit or add a promotion (or finalise a plan), it **snaps the depth to a valid ladder** (0, 5, 10, 15, 20, 30, 40%), **caps the vendor funding** to a sensible share of revenue, and **re-estimates** the units/revenue/profit for the affected lines. It is a fast, consistent read on *your* change.

It is deliberately **not** a re-optimisation: it does **not** re-run the solver, so it won’t re-balance every other item against your edit, and it does **not** re-run the demand model from scratch. That is the point — you keep control of your chosen tactics, and the tool gives you a quick, consistent estimate of them without silently re-deciding the rest of the plan. Re-optimising the whole plan *around* your fixed choice is a separate, heavier operation you can request explicitly.

ROADMAP / LATEST BUILD — Double-hoop promotions

Today the tool models digital-coupon offers with a minimum-buy qualification (one ‘hoop’). Full **double-hoop** mechanics — where a shopper must both clip a digital coupon **and** meet an in-store buy criterion to earn the deal — are handled in the latest build and not yet covered here.

9. The One-Paragraph Version

Every promotion this tool proposes is one that has actually been run; it does not invent depths or tactics, and nothing crosses divisions. It predicts units and revenue, computes profit from real cost, and trusts steady items more than erratic ones. It then chases the goal you choose while floors protect the other two and a set of guardrails — discovered from the data and reflecting established merchandising practice — keep the plan from over-discounting, overspending vendor money, undermining price image, or crowding the ad; without them the ‘pure’ answer is one no merchant could execute. It understands that items compete and pair, so it doesn’t double-count cannibalised volume. It hands you the five best plans for each goal (the optimum plus four alternatives) to choose between, and lets you edit and repair your own lines. The problem is genuinely division-scale — hundreds of millions of options — which is why a serious optimiser is needed and why a full run takes hours; the path to faster runs is more parallel computing, not fewer safeguards. It is a joint build, and the numbers, weights and controls are all things we tune together.

Companion reference: Guardrails & Pruning Rules (full per-guardrail weight matrix and learning scope). This guide describes the system as built.